Tenna Noyes

Professor Florence Bacabac

ENGL 3450

3 November 2026

A Research Proposal on Improving API Technical Documentation

Abstract

Software developers and programmers rely on API documentation to use interfaces correctly and efficiently, however, little is understood about which features of these documents actually help them complete their work. This research proposal contains a design for research aiming to answer three questions: (1) What qualities related to completeness, clarity, quality, code-oriented support, and organization do two selected API documents show? (2) Where do software developers and programmers succeed, make mistakes, or become confused when they follow these documents as written? (3) How might those successes and difficulties be explained by the features of the documents? This proposed research consists of a qualitative design performed in three stages. First, two publicly available API documents from different technology companies will be evaluated using a rubric built from the literature. Next, we will obtain IRP approval, then six programmers will attempt to follow one of the documents as written in one on one sessions and will be interviewed afterward on how it went. Finally, the interview data will be coded, categorized, and modeled, then compared against the rubric results. The studies findings may help current, and future, API technical writers create documents that better support the developers and software engineers who rely on them.

Application programming interfaces (APIs) are how software systems communicate with one another, and the software developers and programmers who build with them rely on clear and accurate documentation to do their jobs correctly and efficiently. Reading documentation is a major part of a software engineer's workload, and Assefi notes that it is not always a simple task, since API documentation can be anything from a single HTML page to a complex, multi-page body of detailed information. API technical writers are the ones who create these documents, and when they do it well, the whole team benefits. Even so, little research watches developers use real API documents and connects what happens to the way those documents were written.

Particularly poorly understood, as later discussed, is how specific features of API documents shape what developers actually do with them.

This research proposal will address the question on how API technical writing documents can be improved so that the software developers and programmers who rely on them can complete their tasks more accurately, quickly, and efficiently. It does so through three smaller questions: What qualities related to completeness, clarity, quality, code-oriented support, and organization do two selected API documents show? Where do developers succeed, make mistakes, or become confused when they follow these documents as written? How might those successes and difficulties be explained by the features of the documents? A review of related and relevant literature will cover the purpose of API technical writing, what works in API technical writing, and how those skills are applied in the workplace, before closing with directions for further research. Afterwards the proposed study seeks to answer these questions using a qualitative approach that pairs a document analysis with one on one, task-based interviews.

The boundaries of the study are intentional. It covers two publicly available documents and six to eight developers completing a first-use task. It leaves out interviews with API technical writers, documentation workflows, hiring and team structure, automatically generated reference material, and measurements such as eye tracking. For this study, API documentation means the written materials, such as guides, tutorials, and references, that explain how to use an API, and developers means software engineers and programmers who use APIs. Engagement is a user's attention to a document, while performance is whether a user can complete a task accurately and efficiently. The results matter to API technical writers, their teams, and the companies that publish APIs.

Literature Review

This literature review examines six sources: an interview of an API technical writer, two book reviews, two empirical studies, and a study on extracting information from API documentation. This review will first go over the purpose and importance of API technical writers. It then examines the strategies that best hold readers' attention and support their efficiency, and how to apply those strategies effectively in the field of API technical writing. It closes by indicating where further research is needed.

The Purpose of API Technical Writing

API technical writers have the important role of creating the documents that provide developers and software engineers with the information they need to use new interfaces and coding languages. However, explaining code is not their sole role. Being an API technical writer requires understanding how an application or interface functions, being able to translate that knowledge for their technical audience of software developers and programmers. Bohm's interview with Joanna Suau of Infobip goes over the vitality of this skillset. Joanna Suau started her academic career in English literature and culture, only later earning a degree in technical writing after taking interest in the API technical writing field after she explored how applications function. With this attraction to the technical she dove into learning programming languages such as JavaScript and researching CSS and HTML. These two skillsets set her apart from other technical writers, leading to a successful career in the API technical writing field (Bohm). Her path stands as a source of inspiration and guidance for future API technical writers, showing that taking part in both the writing aspect and the technical early on works as a huge asset.

Romaine's review of Documenting APIs: Writing Developer Documentation for Java APIs and SDKs by James F. Bisso and Victoria Maki further establishes the roles of an API

technical writer. The review states the beginning of the book dives into key concepts such as interfaces and other computer processes, advising aspiring API writers to further research computer sciences and become acquainted with such interfaces and computer processes. Romaine's review also describes how the book goes into how API technical writers contribute to a team and how to do so effectively, and how to get hired as a technical writer. Overall, Romaine highly praises the book, "Most technical writing curricula don't offer enough coursework that gets into the details of software documentation. That's a shame. If more students read a textbook such as this one, the next generation of technical writers would be more bulletproof against layoffs and outsourcing" (Romaine). These sources reinforce the idea that API technical writers are vital to a team, and can make or break a project.

The importance of the role is further reinforced by research on what software developers and programmers want from API technical writers. Meng et al. surveyed and interviewed software developers about API documentation in their study Application Programming Interface Documentation: What Do Software Developers Want? They found that, "...results show that developers initially try to form a global understanding regarding the overall purpose and main features of an API, but then adopt either a concepts-oriented or a code-oriented learning strategy that API documentation both needs to address. Our results also show that general quality criteria such as completeness and clarity are relevant to API documentation as well. Developing and maintaining API documentation therefore need to involve the expertise of communication professionals" (Meng et al.). The priorities found in this study define the purpose of an API technical writer: producing documents that are accurate, clear, practical, and code-oriented for the developer audience who rely on it. Without API technical writers and this important task, it would greatly hinder the team.

What Works in API Technical Writing

There are several practices that can greatly improve the quality of a document and create engagement with the users. These practices are vital because reading documentation is a major part of a software engineer's workload. In his work on API documentation, Assefi states, "Software engineers need to spend a significant amount of time and effort reading and understanding the requisite documentation. Moreover, this is not always a simple task since API documentation from a provider can be anything from a single HTML page to a complex elaboration of detailed information spanning several pages" (Assefi). This shows the important impact that API technical writers have on the workload of their team. Developers value API documents that are well put together, easy to work with, and quick to understand. The findings of Meng et al. are probably the most direct and useful. Developers value documents that are complete and clear, and they found better results from documentation that uses code-oriented language, code-based explanations, and examples, which helped users better understand and engage with the text, learning faster and more efficiently (Meng et al.).

Badurrek and Meng approach this question from a different angle. Badurrek's study does not mention API technical writers, however it primarily focuses on software tutorials, which is a genre that API technical writers are present in. The study used eye tracking to test whether simplified user interfaces (SUI) helped users understand the document better and work more effectively with a tutorial than an unedited screenshot. SUI affected attention retention, however, it did not improve the work/performance of the users (Badurrek and Meng). This suggests that visual design choices in API technical documents can hold users' focus and attention without actually improving efficiency.

Joanna Suau's interview adds a valuable perspective to the question of efficiency and what works for API writers. Her experience in the field reflects an emphasis on researching and understanding computer science and development, and her advice to newcomers points toward continued technical learning (Bohm). Her interview suggested that the most effective API documentation comes from writers who understand both the subject and the audience they are writing for.

Applying API Technical Writing Skills

Now that the findings have been laid out, it is important to know how to properly use them. Meng et al. offer a set of priorities for API technical writers: quality, clarity, completeness, and code-oriented learning. API technical writers and their teams can use these as a sort of checklist when creating and reviewing a document. Badurrek and Meng's results act as a caution here, as well. Because attention and performance are two separate outcomes of a document, the document in question should be evaluated on whether readers can accomplish their task effectively and quickly, and not only by whether they stay engaged in the document.

Borden's review of APIs: A Strategy Guide by Daniel Jacobson, Greg Brail, and Dan Woods adds a dimension of organization to documentation. Borden explains that the book goes into why companies use APIs and when they need them, along with when it is recommended to assemble an API team, such as when a company must build a mobile application or make its data available to the public. While the review does not mention API technical writers by name, it is still useful for them to know this sort of information. API writers who understand why organizations use APIs, what an API team looks like, and how hiring managers think are better prepared when they are hired into one of these teams, and they are better able to show their value and fit into the team (Borden).

Summary

The six sources examined in this review describe the combined technical knowledge and ability needed to carefully and attentively guide readers through API technical documents. Bohm's interview and Romaine's review show what API technical writers do and how they are able to enter the field successfully. Meng et al. identify what developers value most in API documentation, and Badurrek and Meng show that design choices in a document affect attention while not necessarily improving performance. Borden's review places all of this information in the context of how organizations build and depend on APIs and their API technical writers. Overall, these sources show that effective API technical writers depend on three indisputable building blocks: an understanding of the technology being documented, a focus on the qualities developers value in an API document, and an awareness of their team and organization. Clarity, quality, completeness, and code-oriented learning were the most repeated, and therefore some of the most important, priorities.

There are also areas where further research is indicated. Several of the sources are pointed indirectly at API technical writers, and the field would likely benefit from more research that studies API documentation directly. Meng et al. tell us what developers say they value, and Badurrek and Meng show that attention is not the same as efficiency, but none of the sources observes developers using a specific API document and ties what happens to how that document was written. One recommendation is to study real API documents and the developers who use them, in order to better understand how these documents can be adjusted to improve efficiency, clarity, and so on.

Research Design and Methods

The original research proposed here is a qualitative study intended to better understand what works and what doesn't work in API technical writing documents. Specifically the goal of this proposed research will be to answer these three research questions:

1. What qualities related to completeness, clarity, quality, code-oriented support, and organization do two selected API documents show?
2. Where do software developers and programmers succeed, make mistakes, or become confused when they follow these documents as written?
3. How might those successes and difficulties be explained by the features of the documents, and how can the documents be improved?

Research Question 1 (RQ1) will primarily be answered through a document analysis using a rubric. Research Question 2 (RQ2) will be answered through one on one sessions in which developers attempt to follow a document as written and are then interviewed. Finally, Research Question 3 (RQ3) will be answered by comparing the results with the coded interview data. I chose qualitative research because these questions ask where, and why, developers succeed or struggle, which calls for quality descriptions more than numbers.

Data Collection

Step 1: Selecting the Documents

First I will select two open sourced API technical documents from different technology companies. A possible technology company is GitHub. Both will be publicly available and will describe a task that a developer can attempt in one sitting, such as authenticating and making a

first request. Choosing documents from different companies allows different approaches to be compared. Keeping the tasks similar prevents differences in difficulty from getting in the way.

Step 2: Task Based Sessions and Interviews

Because the study involves interviewing programmers and software engineers, I will first submit it to the Institutional Review Board (IRB) so that the board can decide whether it qualifies for exemption or needs review. Participants will give informed consent and will be identified only by codes. The document analysis alone would not need IRB approval because the documents are public. Each interviewee will meet with the interviewer one on one, go through a document, and attempt to follow its instructions as written without outside help. I will record the steps completed, mistakes, and points of hesitation. Afterward, the interviewer will ask interview questions such as, what was your first impression of this document? Where did you feel confident, and where did you feel like you struggled? Did you rely more on the explanations or on the code examples/images? Was any information missing or hard to find or understand? In your experience, where do mistakes or confusion happen most often with documents like this? Sessions will be recorded with permission and written down.

Sampling

I will use purposeful sampling for both the API documents and the participants. Participants will be six to eight software engineers or programmers with different levels of experience who have worked with APIs but not the specific ones in this study, so they come to the documents as new users with fresh insights. I will recruit through campus contacts, and online developer communities. This sample is not meant to represent every programmer or software engineer, or to support statistical inference. It is meant to provide detailed accounts from the people who use API documents most.

Data Analysis

For the document analysis I will score and annotate each document using a rubric built from the priorities in Meng et al.: completeness, clarity, quality, and code-oriented learning support, plus organization and whether the document helps a reader form an overall understanding of the API's purpose and main features. Specific passages will be noted as evidence and the two documents will be compared on each criterion. For the interviews and observation notes, I will use coding, categorizing, and modeling. First I will assign codes to segments of data, such as; 'missing prerequisite step', or 'helpful code example'. Then I will group the codes into categories and build a model showing how document features relate to the successes and difficulties participants had. Finally I will compare the interview results with the rubric to see if the problems participants described match the features the rubric found. Following Badurrek and Meng's warnings, I will focus on whether participants completed tasks accurately and efficiently, not only on whether they found a document engaging or not. To strengthen reliability, I will ask a peer or my instructor to review a sample of my coding.

Preliminary Implications

This research should add to and build upon the existing body of study on API technical writing. The literature shows what developers say they want, and that attention and performance are separate outcomes, but it does not observe developers using specific API documents. This study should also test whether the priorities identified by Meng et al. hold up when developers use real documents and it applies Badurrek and Meng's distinction to API documentation directly.

As a result of this research, API documentation will be improved to better work with the software developers and programmers who use these documents. This research is intended to understand what does and does not work in API technical documents, so it will help current and future API technical writers make improved documents. How API technical writers create these documents may also change, using methods that are more effective than techniques previously used.

The results of this research may change how API technical documents are perceived, organized, and written; and with that, how the practitioners who read and write them interact with them. It will mean that practitioners in this setting have an easier time understanding and using these documents. API technical writers and their teams could use the findings as a research backed checklist when creating and reviewing a document, one that asks whether readers can finish their task quickly and correctly and not only whether they stay engaged. Technical writing programs, which Romaine says often lack coursework on software documentation, could also benefit.

There are, of course, limitations to this study. Only studying two documents would be a significant limitation, because, the findings may not apply to all API documentation. The small, purposefully chosen sample also limits how far the results can be generalized, being observed may change how participants behave, and the analysis depends on my own judgment. To address these weaknesses, I will use clear rubric criteria, compare the document analysis with the interview findings, and have a second reader check part of my coding. It would be advised that future research be performed on more API technical documents, with a larger and more varied group of documents and users, and possibly with quantitative measures such as time on task or eye tracking.

Conclusion

API technical writing is an incredibly specialized and unique subsection of technical writing, but the work that API technical writers do is felt by every developer, software engineer, and programmer who opens an unfamiliar interface and needs to get something going. This study should be done because most research on API documentation focuses on what developers say they want or on related genres, rather than on observing real documents in use. Its specific purpose is to answer how API technical writing documents can be improved by examining what two documents contain and how developers succeed or struggle when following them. I chose a qualitative design that combines a document analysis with task based interviews because it connects what developers do with why they do it, and ties both back to the text.

The potential implications are evidence based guidance for writers and a practical checklist for team review. Within the broader scholarship, this study builds on Meng et al. and on Badurrek and Meng, moving both toward the direct study of API documentation that the field still needs.

Works Cited

Assefi, Mehdi. “Towards API Documentation Information Extraction: Challenges and Approaches.” 2021. ProQuest, www.proquest.com/openview/6c72baabbec5a819273f797d1b7d1523/1?cbl=18750&diss=y&pq-origsite=gscholar.

Badurrek, P., and M. Meng. “Simply Effective? Simplified User Interfaces in Software Tutorials.” Journal of Technical Writing and Communication, vol. 55, no. 3, 2024, pp. 223–46. https://doi.org/10.1177/00472816241262221.

Bohm, T. “An Interview with Joanna Suau from Infobip on the Design of Application Programming Interface (API) Documentation.” Information Design Journal, vol. 26, no. 2, 2021, pp. 91–104. https://doi.org/10.1075/idj.00011.boh.

Borden, Mary C. “APIs: A Strategy Guide.” Technical Communication, vol. 60, no. 2, May 2013, pp. 154–55. EBSCOhost, research.ebsco.com/plink/ba671547-183f-3f81-b187-706d13ed392a.

Meng, M., et al. “Application Programming Interface Documentation: What Do Software Developers Want?” Journal of Technical Writing and Communication, vol. 48, no. 3, 2017, pp. 295–330. https://doi.org/10.1177/0047281617721853.

Romaine, Garret. “Documenting APIs: Writing Developer Documentation for Java APIs and SDKs James F. Bisso Victoria Maki.” Technical Communication, vol. 54, no. 2, May 2007, pp. 233–35. EBSCOhost, research.ebsco.com/plink/745d11d7-0ba8-32f8-9170-18203abf2014.